JavaScript
some “advanced” topics

Tvorba internetových aplikácií

RNDr. Richard Ostertág, PhD.
KI FMFI UK Bratislava
ostertag@dcs.fmph.uniba.sk

Sources

- Mozilla Developer Network: JavaScript
  - https://developer.mozilla.org/en-US/docs/JavaScript
- JSLint
  - http://www.jslint.com/
- Annotated ECMAScript 5.1
  - http://es5.github.com/
- ECMAScript Language Specification
  - Standard ECMA-262 / 5.1 Edition / June 2011
  - http://ecma-international.org/ecma-262/5.1/

Regular expressions

- similar to Perl regular expressions
- following notations are equivalent
  - var re = new RegExp("\\d+$", "m");
  - var re = /\d+$/m;
- var re = /a(b+)/g; var str = "abbcacabcb"; var a;
  while ((a = re.exec(str)) !== null)
    console.log("Found", a[0], a[1],
      "next search from", re.lastIndex);
  - Found abb bb next search from 3
  - Found ab b next search from 8

Regular expressions – replace

- var re = /^\s*(\w+)\s+(\w+)\s*$/;
- var str = " John   Doe ";
- str.replace(re, "$2, $1");
  - "Doe, John"
- str = "Apple is not fruit, Apple is corporation.";
- str.replace(/apple/gi, "Orange");
  - "Orange is not fruit, Orange is corporation.“
- str.replace("apple", "Orange", "gi");
  - "Orange is not fruit, Orange is corporation."

Typed arrays

- typed arrays are split into
  - buffer
    - represents a chunk of data
    - no format (type of elements)
    - no mechanism for accessing its contents
    - implemented by the ArrayBuffer class
  - view
    - provides elements type, starting offset and number of elements
    - implemented by the ArrayBufferView subclasses
- typed arrays can have multiple views

Typed arrays – cont.

- var buf = new ArrayBuffer(16);
  - [object ArrayBuffer]
  - chunk of 16 bytes all initialized to 0
- buf.byteLength // 16
- var int32Vw = new Int32Array(buf);
  - ({0:0, 1:0, 2:0, 3:0})
- for (var i = 0; i < int32Vw.length; i++) int32Vw[i] = 2 * i;
  - ({0:0, 1:2, 2:4, 3:6})
- var int16Vw = new Int16Array(buf, 2, 6);
  - buffer, offset (in bytes, div. by elem. size), length (in elements)
- for (var i = 0; i < int16Vw.length; i++)
  console.log("int16Vw[" + i + "]=" + int16Vw[i]);
  - int16Vw[0]=0, int16Vw[1]=2, int16Vw[2]=0,
    int16Vw[3]=4, int16Vw[4]=0, int16Vw[5]=6

Typed arrays – init.

- initialization of typed array is done via view
  - var byteArray = new Uint8Array([1, 2, 3, 4]);
    - ({0:1, 1:2, 2:3, 3:4})
- underlying ArrayBuffer can be retrieved
  - byteArray.buffer
    - [object ArrayBuffer]
- conversion of typed array to normal array
  - var normalArray = Array.apply([], byteArray);
    - [1, 2, 3, 4]

Typed arrays – DataView

- provides a low-level interface for reading unaligned data with various byte order.
- var buf = (new Int32Array([1, 2, 4, 6])).buffer;
  - ({0:0, 1:2, 2:4, 3:6})
- var int16Vw = new Int16Array(buf, 1, 6);
  - Error: invalid arguments
- var dataVw = new DataView(buf, 1, 6 * 2);
  - [object DataView]
- for (var i = 0; i < 6; i++) console.log(dataVw .getInt16(i /*byte offset*/, true /* big-endian*/));
  - 0, 0, 512, 2, 0, 0

Array comprehensions

- requires JavaScript ≥ 1.7
- are used to construct a new array based on the another array
- often can replace
  - Array.map()
  - Array.filter()

Array comprehensions – map

- var nums = [2, 5, 4, 7]; var sqrs;
- beware of the difference between **in** and **of**
- sqrs = [x * x for (x **in** nums)];
  - [0, 1, 4, 9]
- sqrs = [x * x for (x **of** nums)];
  - [4, 25, 16, 49]
- nums.map(function (x) {return x * x;});
  - [4, 25, 16, 49]

Array comprehensions – filter

- var nums = [2, 4, 5, 7]; var evens;
- beware of the difference between **in** and **of**
- evens = [x for (x **in** nums) if (x % 2 === 0)];
  - ["0", "2"]
- evens = [x for (x **of** nums) if (x % 2 === 0)];
  - [2, 4]
- nums.filter(function (x) {return x % 2 === 0;});
  - [2, 4]

Array comprehensions – comb.

- var nums = [2, 4, 5, 7]; var list;
- beware of the difference between **in** and **of**
- list = [x * x for (x **in** nums) if (x % 2 === 0)];
  - [0, 4]
- list = [x * x for (x **of** nums) if (x % 2 === 0)];
  - [4, 16]
- nums.filter(function (x) {return x % 2 === 0;})
  .map(function (x) {return x * x;});

Array comprehensions – strings

- var str = "richard";
- var sm = [String.fromCharCode(c.charCodeAt(0) + 1) for (c of str)].join("");
  - "sjdibse"
- var plain = [String.fromCharCode(c.charCodeAt(0) - 1) for (c of sm)].join("");
  - "richard"

Iterators

- requires JavaScript ≥ 1.7
- iterator is an object that with next() method
  - returns the next element from the sequence
  - raise a StopIteration exception
    - when sequence has no next element
- iterator object can be used
  - explicitly by repeatedly calling next()
  - implicitly using for ... in and for each constructs

Custom iterators

- function Range(low, high) { this.low = low; this.high = high; }
  - custom Range object which stores a low and high value
- function RangeIterator(range)
  { this.range = range; this.current = this.range.low; }
  - custom RangeIterator object which stores iterator state
- RangeIterator.prototype.next = function ()
  { if (this.current > this.range.high)
  throw StopIteration;
  else
  return this.current++; };
  - custom RangeIterator object next() method
- Range.prototype.__iterator__ =
  function () { return new RangeIterator(this); };
  - associate RangeIterator with the Range object via special __iterator__ method
- var range = new Range(3, 5);
  for (var x in range) console.log(x); // logs 3, 4, 5

Generators

- requires JavaScript ≥ 1.7
- generator is a function containing at least one yield expression
  - returns generator object
    - works as a factory for iterators
- enable lazy computation of sequences
  - elements calculated on-demand
  - automatically maintain its own state

Simple generator

- function gen() { yield "a"; yield "b";
  for (var i = 1; i < 3; i++) yield i; }
- var g = gen(); // [object Generator]
- console.log(g.next()); // a
- console.log(g.next()); // b
- console.log(g.next()); // 1
- console.log(g.next()); // 2
- console.log(g.next()); // [object StopIteration]
- console.log(g.next()); // [object StopIteration]

Replacing iterator with generator

- function Range(low, high)
  { this.low = low; this.high = high; }
  - custom Range object – stores a low and high value
- Range.prototype.__iterator__ = function ()
  { for (var i = this.low; i <= this.high; i++) yield i; };
  - associate generator with the Range object
    (via special __iterator__ method)
- var range = new Range(3, 5);
  for (var x in range) console.log(x); // logs 3, 4, 5

Generator – infinite Fibonacci seq.

```
function fib() {
  var fn_2 = 1;
  var fn_1 = 1;
  while (true) {
    var tmp = fn_2;
    fn_2 = fn_1;
    fn_1 = fn_1 + tmp;
    yield tmp;
  }
}
```

- var f = fib();
  - [object Generator]
- console.log(f.next()); // 1
- console.log(f.next()); // 1
- console.log(f.next()); // 2
- console.log(f.next()); // 3
- console.log(f.next()); // 5

Generator – finite Fibonacci seq.

- function fib(max) {
  var fn_2 = 1;
  var fn_1 = 1;
  while (true) {
  var tmp = fn_2;
  fn_2 = fn_1;
  fn_1 = fn_1 + tmp;
  if (max && tmp > max)
  return;
  yield tmp;
  }
  }

- var f = fib();
  - does the same as prev.
- var f = fib(5);
  - [object Generator]
- console.log(f.next()); // 1
- console.log(f.next()); // 1
- console.log(f.next()); // 2
- console.log(f.next()); // 3
- console.log(f.next()); // 5
- console.log(f.next());
  - [object StopIteration]

Generator expressions

- requires JavaScript ≥ 1.8
- array comprehensions construct an entire new array in memory
  - unfeasible if array is large or even infinite
- generator expressions are lazy
  - e.g. can be applied even to infinite sequences
- almost identical to array comprehensions
  - they use parentheses ‘()’ instead of braces ‘[]’

Generator expressions

- function fib(max) {
  var fn_2 = 1;
  var fn_1 = 1;
  while (true) {
  var tmp = fn_2;
  fn_2 = fn_1;
  fn_1 = fn_1 + tmp;
  if (max && tmp > max)
  return;
  yield tmp;
  }
  }

- var f = fib(21);
  - [object Generator]
- var a = [x for (x in f)];
  - [1, 1, 2, 3, 5, 8, 13, 21]
- var f = fib();
- var iter = [x for (x in f)];
  - will kill your browser!
  - but wait for it: NS_ERROR_OUT_OF_MEMORY
- var g = (x for (x in f) if (x % 2 === 0));
  - [object Generator]
- for (var i = 0; i < 20; i++) console.log(g.next());

Transpilers

- a source-to-source compiler
- main JavaScript.next to JavaScript-of-today compilers:
  - Traceur (by Google)
  - Babel (formely 6to5)
    - more readable code
    - supports JSX (JavaScript syntax extension similar to XML)
    - doesn't need any runtime extra script to run
- try it online:
  - https://google.github.io/traceur-compiler/demo/repl.html
  - https://babeljs.io/repl/

ECMAScript support

- ECMAScript 6
  - http://kangax.github.io/compat-table/es6/
  - Traceur: 61%
  - Babel: 73%
- ECMAScript 7
  - http://kangax.github.io/compat-table/es7/
  - Traceur: 10%
  - Babel: 67%

Traceur

```
function f(n) {
  if (n <= 1) return 1;
  return f(n - 1)
}
function* gen() {
  yield "a";
  yield "b";
  for (var i = 1; i < 3; i++) yield i;
}
var g = gen();
console.log(g.next());
```

```
$traceurRuntime.ModuleStore.getAnonymousModule(functio
n() {
  "use strict";
  var $__3 = $traceurRuntime.initTailRecursiveFunction(f),
      $__5 = $traceurRuntime.initTailRecursiveFunction(gen);
  var $__1 = $traceurRuntime.initGeneratorFunction(gen);
  function f(n) {
    return $traceurRuntime.call(function(n) {
      if (n <= 1)
        return 1;
      return $traceurRuntime.continuation(f, null, [n - 1]);
    }, this, arguments);
  }
 function gen() {
    return $traceurRuntime.call(function() {
      var i;
      return
$traceurRuntime.continuation($traceurRuntime.createGenera
torInstance, $traceurRuntime,
[$traceurRuntime.initTailRecursiveFunction(function($ctx) {
        return $traceurRuntime.call(function($ctx) {
while (true)
          switch ($ctx.state) {
            case 0: $ctx.state = 2; return "a";
            case 2: $ctx.maybeThrow(); $ctx.state = 4; break;
            case 4: $ctx.state = 6; return "b";
            case 6: $ctx.maybeThrow(); $ctx.state = 8; break;
            case 8: i = 1; $ctx.state = 15; break;
            case 15: $ctx.state = (i < 3) ? 9 : -2; break;
            case 12: i++; $ctx.state = 15; break;
            case 9: $ctx.state = 10; return i;
            case 10: $ctx.maybeThrow(); $ctx.state = 12; break;
            default:
              return $traceurRuntime.continuation($ctx.end, $ctx, []);
          }
        }, this, arguments);
      }), $__1, this]);
    }, this, arguments);
  }
  var g = gen();
  console.log(g.next());
  return {};
});
//# sourceURL=traceured.js
```

Babel

```
function f(n) {
  if (n <= 1) return 1;
  return f(n - 1)
}
function* gen() {
  yield "a";
  yield "b";
  for (var i = 1; i < 3; i++) yield i;
}
var g = gen();
console.log(g.next());
```

```
"use strict";

var marked0$0 = [gen].map(regeneratorRuntime.mark);
function f(_x) {
  var _again = true;

  _function: while (_again) {
    var n = _x;
    _again = false;

    if (n <= 1) return 1;
    _x = n - 1;
    _again = true;
    continue _function;
  }
}
```

```
function gen() {
  var i;
  return regeneratorRuntime.wrap(function gen$(context$1$0)
{
    while (1) switch (context$1$0.prev = context$1$0.next) {
      case 0: context$1$0.next = 2; return "a";
      case 2: context$1$0.next = 4; return "b";
      case 4: i = 1;
      case 5:
        if (!(i < 3)) { context$1$0.next = 11; break; }
        context$1$0.next = 8;
        return i;
      case 8:
        i++; context$1$0.next = 5; break;
      case 11:
      case "end":
        return context$1$0.stop();
    }
  }, marked0$0[0], this);
}
var g = gen();
console.log(g.next());
```